

MT5 AI Trading Bot — Technical Documentation

Autonomous, AI-assisted MetaTrader 5 trading platform. A Fastify/TypeScript backend drives a Python MT5 bridge, reasons over market data with a pluggable LLM, enforces a pure risk engine as the final authority on every order, and surfaces everything through a Next.js dashboard plus Telegram/WhatsApp connectors.

Last updated: 2026-06-19. This document describes the system as built; it is the canonical architecture reference. For the HTTP contract see [API.md](#).

1. What this system is

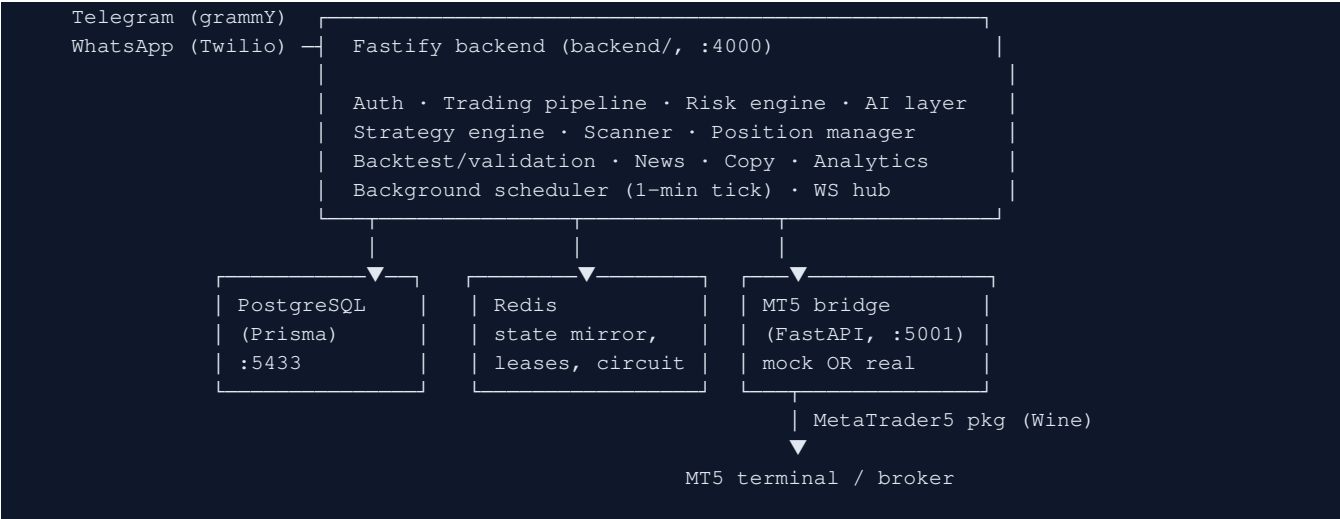
A self-hosted trading bot that:

- Connects to a **MetaTrader 5 terminal** (live or demo) through a small Python HTTP **bridge**.
- Continuously analyzes a watchlist of FX/metal symbols with classical technical indicators.
- Uses an **LLM (Ollama/Anthropic/OpenAI/OpenRouter)** as an *advisory* reasoning layer that can veto but never force a trade.
- Runs every proposed order through a **pure, exhaustively-tested risk engine** that is the final authority.
- Operates in four modes — **MANUAL** (recommend only), **SEMI_AUTO** (human approves), **AUTO** (fully autonomous), **COPY** (mirror an external trader).
- Manages open positions (break-even, ATR trailing, time-exit, news flatten, equity guardian).
- Provides backtesting, walk-forward, Monte-Carlo and out-of-sample validation so strategies are vetted before they trade.

⚠ **Honest note on edge.** Extensive in-repo backtesting (see [backtest-forensic-audit-2026-06-15.md](#)) has not found a robust, deployable edge in the bundled confluence / mean-reversion / breakout strategies on majors and gold. The platform's value is its *discipline and infrastructure* (validation, risk control, observability), not a guaranteed profitable signal. Loosening gates makes the bot trade; it does not make it profitable.

2. Architecture





Process model (typical dev/live setup on the user's Mac):

Process	How it runs	Port
PostgreSQL	Docker Compose	5433 (host)
Redis	Docker Compose	6379
MT5 bridge	python main.py under Wine, attached to the MT5 terminal	5001
Backend	npm run dev (tsx watch)	4000
Frontend	npm run dev (next)	3000
Ollama (default AI)	local daemon	11434

Only Postgres + Redis live in Docker; backend/frontend/bridge run on the host.

3. Technology stack

- Backend** — Node.js + TypeScript (ESM), Fastify, Prisma ORM (PostgreSQL), ioredis, @fastify/jwt + bcryptjs + otplib (auth/2FA), grammY (Telegram), zod (validation), pino (logging), vitest (tests).
- Bridge** — Python 3.12, FastAPI + uvicorn + pydantic, official MetaTrader5 package (real mode) or a built-in random-walk mock (MT5 MOCK=true).
- Frontend** — Next.js (App Router) + React 19, Tailwind v4 (@theme tokens, OLED dark fintech theme), native WebSocket, vitest.
- Infra** — Docker Compose (Postgres, Redis), Dockerfile.real for a Wine + Windows Python + MT5-terminal container.

4. Repository layout

```
backend/
src/
```

index.ts	Fastify bootstrap, plugin + route registration, shutdown
config.ts	zod-validated env (fail-closed in production)
lib/	prisma, redis, logger, audit, crypto, resilience, operational-health
modules/	
auth/	JWT login/register, 2FA (TOTP), token revocation, role plugin
mt5/	bridge client (circuit-breaker), account registry, symbol remap, routes
analysis/	indicators + market-analysis engine (multi-timeframe)
strategy/	signal generation, presets, AI Strategy Lab, symbol expansion, validation
ai/	provider abstraction (ollama/anthropic/openai/openrouter), schema, prompts
risk/	PURE risk engine, instrument value model, exposure gate
trading/	THE pipeline: service, scanner, manager, paper, reconciliation, symbol-lock, day-trading, floating-pnl, execution-comparison, routes
news/	economic calendar + RSS headlines, news gate, pre-news flatten
backtest/	engine, execution sim, metrics, monte-carlo, validation, market-data
copy/	copy-trading service + signal parser
notifications/	multi-channel dispatch (web/telegram/whatsapp/email)
telegram/ whatsapp/	chat connectors
system/	bot state machine, system/health/audit routes
incidents/	dedupe-keyed incident store + routes
sentiment/ journal/	analytics/ web/ supporting features
ws/	WebSocket hub (broadcast)
workers/	
scheduler.ts	timers; protective tick + new-trade tick
scheduler-cycle.ts	coordinated cycle (protect always, new-trade under lease)
scheduler-lease.ts	Redis lease wrapper (single-writer across processes)
prisma/schema.prisma	full data model
mt5-bridge/main.py	FastAPI bridge (mock + real brokers)
frontend/	Next.js dashboard
docs/	API.md, this file, forensic audit
docker-compose.yml	Postgres + Redis

5. The trade lifecycle (core pipeline)

`evaluateAndMaybeTrade(user, strategy, symbol)` in `trading/service.ts` is the **only** path that can open a strategy-driven position. Every stage is recorded in the trade's `explanation` JSON for a full decision trail.

0. Platform/operational gates	— emergencyStop? status==running? Redis available?
1. Symbol lock	— another strategy/manual/scanner trade already holds this market (scoped to current account)? stand aside.
2. Market data + analysis	— tick + candles per timeframe → <code>buildMarketAnalysis()</code> (gated by "new completed bar" checkpoint per strategy×symbol)
3. News gate	— <code>assessNewsRisk()</code> : allow reduce pause
4. Strategy signal	— <code>evaluateStrategy()</code> : direction + reasons, or no_signal
5. AI reasoning (ADVISORY)	— <code>askModel()</code> : must AGREE with signal AND meet minConfidence, else <code>ai_veto</code> (it can veto, never force). Invalid/unreachable AI = <code>ai_unavailable</code> (distinct from "avoid").
6. Position sizing	— <code>calculateLots()</code> from risk% + stop distance + instrument spec
7. Risk engine (FINAL)	— <code>validateTrade()</code> : 21 checks; on fail → RISK_BLOCKED trade row
8. Paper-forward?	— if on, open a simulated PaperTrade instead of a broker order
9. Mode gate:	
MANUAL	→ ANALYZED row + "recommendation" notification (no order)
SEMI_AUTO	→ PENDING_APPROVAL row + approval (TTL 15 min); human picks lot size
AUTO	→ <code>executeTrade()</code> immediately (broker order)

Approval path (`decideTrade`): re-runs the **full** risk check with fresh market data at approval time — the approver-chosen lot size still goes through the engine; the engine has the final word and can re-block.

Execution (`executeTrade`): sends the order via the bridge, stores `mt5Ticket = position_id ?? order_ticket` (position id is what reconciliation/close/modify match on — correct on netting accounts), records an **execution comparison** (expected vs actual entry, slippage, latency), audits, notifies, and

The **scanner** ([trading/scanner.ts](#)) is a parallel autonomous path with the same gate philosophy — see §8.

6. Risk engine (final authority)

[risk/engine.ts](#) — **pure, no I/O**, so it is unit-tested exhaustively. `validateTrade(proposal, ctx)` runs every check and a trade proceeds only if **all** pass. Checks:

Group	Checks
Platform	<code>emergency_stop</code> , <code>bot_running</code>
Live gating (live accounts only)	<code>live_settings_enabled</code> , <code>live_user_enabled</code> , <code>live_2fa</code> (demo accounts skip these)
News / exposure	<code>news</code> (pause = block), <code>exposure</code> (per-currency & correlated caps)
Stops	<code>stop_loss_required</code> , <code>take_profit_required</code> , <code>stop_loss_direction</code> , <code>min_risk_reward</code>
Sizing	<code>max_risk_per_trade</code> (money-at-risk via instrument value model; tolerance $\times 1.5$), <code>max_lot_size</code>
Exposure counts	<code>max_open_trades</code> , <code>max_trades_per_symbol</code> , <code>max_trades_per_day</code> , <code>max_consecutive_losses</code> (circuit breaker)
Copy	<code>max_daily_copied_trades</code> , <code>copy_exposure_limit</code>
Loss limits	<code>max_daily_loss</code> , <code>max_weekly_loss</code>
Drawdown	<code>max_drawdown</code> (from peak), <code>equity_protection</code> (equity floor vs balance)
Market	<code>max_spread</code> , <code>volatility_limit</code> (ATR%)
Session	<code>trading_session</code> (london/newyork/etc.)

Notable behaviors:

- **News "reduce"** halves the lot size instead of blocking (`adjustedLots`).
- **max_risk_per_trade** uses a real money model (`moneyForPriceMove / valuePerPointPerLot` in [risk/instruments.ts](#)) correct for FX majors, JPY pairs, metals, indices and crypto — and a **$\times 1.5$ tolerance** on the configured percent. On a small account the 0.01-lot minimum can still exceed a tight per-trade cap, which legitimately blocks every trade until the cap is raised or the account funded.
- **Consecutive-loss breaker** (`countConsecutiveLosses`) skips unreconciled (null-profit) closes rather than treating them as wins — an attribution gap can't silently reset it.
- **equityGuardianBreaches** (separate from new-trade gating) drives the position-flattening guardian; it triggers on the equity floor only, never on drawdown-from-peak (which would flatten winners on ordinary pullbacks).

`calculateLots()` sizes from `riskAmount / perLotRisk`, floored to the instrument's volume step and clamped to `[volumeMin, min(maxLot, volumeMax)]`.

7. AI reasoning layer

[ai/service.ts](#) + [ai/providers/](#). Exactly **one** active provider at a time (no silent fallback), switchable at runtime via `PUT /api/ai/provider` (persisted in `SystemSetting`). Providers: `ollama` (default), `anthropic`, `openai`, `openrouter` (OpenAI-compatible).

- `askModel(prompt, symbol)` calls the provider with `SYSTEM_PROMPT` (an institutional trading checklist), parses the JSON response against `AiDecisionSchema` (`decision: buy|sell|hold|avoid`, `confidence 0-1`, `reasoning`, `risk_level`, optional suggested entry/SL/TP, `news_risk`, `should_execute`), and logs **every** call to `AiAnalysisLog` with `valid` + parsed decision.
- Invalid/unparseable/unreachable → `AI_SAFE_FALLBACK` (`avoid`, `conf 0`) and `valid=false`. Distinguished in audit as `ai_unavailable` so a down model is visible rather than looking like a disagreement.
- **Gating**: the AI must *agree with the strategy direction* AND meet the strategy/scanner minimum confidence. It can veto but never originate a trade.
 - In the **scanner**, `aiMode` is `STRICT` (veto blocks) or `ADVISORY` (logged, non-blocking).
 - In `ADVISORY` mode `minAiConfidence` is effectively inert (only consulted in `STRICT`).
- `aiHealth()` reports reachable + model-present + recent valid-rate (from the last 20 `AiAnalysisLog` rows), surfaced on `/health` and the dashboard health badge.

The **AI Strategy Lab** ([strategy/lab.ts](#)) uses the LLM as a *hypothesis generator*: it proposes strategy ideas (constrained schema), each is auto-validated by walk-forward at realistic per-instrument spread, and only survivors are saved as **disabled** candidates. Nothing the AI proposes can trade without passing validation **and** manual enable.

8. Strategy engine & autonomous scanner

Signals — `evaluateStrategy(strategy, analysis)` in [strategy/service.ts](#) supports three entry styles via `config.entry.style`:

- `confluence` (default) — multi-timeframe trend/structure/RSI/MACD/pattern agreement.
- `mean_reversion` — fade Bollinger 2σ extremes + RSI extreme, filtered by higher-TF bias, optional ADX regime filter (`regimeMaxAdx`).
- `breakout` — London-session break of the Asian range.

`deriveLevels()` produces entry/SL/TP (ATR-based). Strategy `config` is JSON: symbols, timeframes, entry/exit rules, lot sizing, sessions, per-strategy risk limits. `ALL_FX` is a sentinel symbol expanded to the broker's FX universe (capped at 40) by [strategy/symbols.ts](#).

Scanner ([trading/scanner.ts](#)) — a strategy-less autonomous sweep configured in `SystemSetting` key `scanner` (`enabled`, `symbols`, `intervalMin`, `maxPerDay`, `minScore`, `aiMode`, `minAiConfidence`):

1. Scores each watchlist symbol with `scoreSymbol()` (confluence across timeframes; max 6 points; requires higher-TF alignment; anti-chasing filter rejects entries >1.5 ATR from EMA20).
2. Keeps candidates with `score ≥ minScore`, best first; skips symbols with a pending suggestion.

4. In **AUTO**: executes (or opens a paper trade if paper-forward is on). Otherwise creates a `PENDING_APPROVAL` suggestion (human chooses the lot size). A directed scan (`opts.symbol`) lowers the confluence bar to 2 and bypasses the daily cap.

The one-position-per-pair **symbol lock** is enforced in the strategy pipeline (`evaluateAndMaybeTrade`), not in the scanner — the scanner can stack multiple positions on the same pair, bounded instead by the exposure caps and `maxPerDay`.

9. Market analysis & indicators

`analysis/engine.ts` builds a `MarketAnalysis` (bid/ask, spread, session, per-timeframe analysis array, reference range for breakouts) from a tick + candles. Each `TimeframeAnalysis` carries trend, RSI (+ previous), MACD line/signal/ histogram (+ previous), EMA fast/slow, Bollinger position, ATR + ATR%, ADX, support/resistance, last close, candle pattern, and market structure (`higher_highs / lower_lows / consolidation`).

Indicators (`analysis/indicators.ts`) are pure functions: `sma`, `ema`, `rsi`, `macd`, `bollinger`, `atr`, `adx`, `last`. Candles are normalized and the pipeline only acts on **completed** bars (a per-strategy×symbol checkpoint in `SystemSetting` prevents re-processing the same bar).

10. Position management

`trading/manager.ts` runs every scheduler tick, **even while paused** (protecting open positions ≠ opening new ones):

- **Break-even** at +1R (stop to entry + 0.1R buffer).
- **ATR trailing** beyond +1.5R (trail one ATR(14,H1) behind price).
- Stops only ever **ratchet** in the trade's favor — never widened.
- **Time-exit** for trades with `closeAfterMin`.
- **External adoption** — positions opened outside the bot are adopted as tracked `Trade` rows (mode `MANUAL`) so break-even/trail/time-exit apply; their open price + current stop are captured once as a stable R reference.

Other protective jobs (scheduler `protectiveTick`): **pre-news flatten** (`news/flatten.ts`), **equity guardian** (flatten + pause when equity breaches the floor), **day-trading exit** (flatten past the daily UTC cutoff), and **closed-trade reconciliation** (login-scoped to the current account).

11. Backtesting & validation

`backtest/` reuses the **same** analysis/strategy/ level-derivation/sizing code as live trading (no separate logic), with next-bar-open execution, stop-first same-bar fills, and spread/slippage/commission modeling.

~~There is no AI layer in backtests (AI is a live-only gate).~~

- `POST /api/backtest` — single run (metrics: return%, win rate, profit factor, R-multiples...).
- `POST /api/backtest/walk-forward` — N consecutive non-overlapping windows of the *same fixed* strategy (not parameter re-optimization), with a `consistency` verdict.
- `monte-carlo.ts`, `validation.ts` — trade-shuffle MC and IS/OOS gates. Validation results persist as `ValidationRun` rows and gate the Strategy Lab.

The methodology (walk-forward + realistic per-instrument spread + parameter-sensitivity sweep

- temporal/instrument OOS) is designed to *catch false positives*, and consistently has.

12. MT5 bridge

`mt5-bridge/main.py` — FastAPI service, API-key protected (`x-api-key` header). Two interchangeable brokers behind one interface:

- **MockBroker** (`MT5 MOCK=true`, default) — random-walk prices, in-memory positions; lets the whole stack run with no terminal.
- **RealBroker** — thin adapter over the official `MetaTrader5` package (Windows/Wine); `connected()` verifies `terminal_info()` + `account_info()`.

Endpoints: `GET /health, /account, /positions, /history, /tick/{symbol}, /symbol/{symbol}, /candles/{symbol}, /symbols`; `POST /connect, /order, /position/{ticket}/modify, /position/{ticket}/close`.

The backend's `mt5/client.ts` wraps these with a **circuit breaker** (failure threshold 3, 30 s cooldown — opens to all symbols on repeated failure and raises an incident) and **symbol remapping** (`matchBrokerSymbol / SYMBOL_ALIASES`) so configured names like `EURUSD` auto-resolve to broker-suffixed forms (`EURUSDm`) and aliases (`XAUUSD↔GOLD`, `NAS100↔USTEC`, ...). `mt5/account.ts` maps the connected login → `Mt5Account` row (upsert by `(userId, login)`), stamping every trade with the account it ran on.

13. Data model (Prisma / PostgreSQL)

Key entities (`schema.prisma`):

Model	Purpose
User	account, role (ADMIN/MANAGER/VIEWER), TOTP, <code>liveTradingEnabled</code>
Mt5Account	connected terminal login (unique per <code>(userId, login)</code>), encrypted password, demo flag
Strategy	name, type, enabled, <code>JSON config</code>
RiskSettings	per-user risk parameters (the engine's inputs)
Trade	the lifecycle record: status, mode, levels, <code>mt5Ticket</code> , profit, explanation (decision trail), links to AI log/approval/comparison/journal

TradeApproval	semi-auto approval with TTL + channel
PaperTrade	simulated fills for paper-forward (cost-modeled)
ExecutionComparison	expected vs actual entry/exit, slippage, latency, P&L attribution
ValidationRun	persisted walk-forward/OOS validation evidence + gates
AiAnalysisLog	every LLM call: prompt, raw + parsed response, decision, confidence, valid
CopyTrader / CopiedTrade	copy-trading profiles, rules, mirrored trades
NewsEvent	economic calendar + classified headlines
Notification	multi-channel outbox
AuditLog / ErrorLog	append-only audit trail + error log
Incident	dedupe-keyed operational incidents (open/ack/resolved)
TradeJournalEntry	post-trade notes/lessons/rating
SystemSetting	key/value JSON: bot_state, scanner, ai_provider, day_trading, per-account peak_equity:*, bar checkpoints, lab reports, token-revocation cutoffs

Trades, P&L aggregation, loss streaks, peak equity and drawdown are all **scoped per account**, so switching the terminal login doesn't let one account's history block or distort another's.

14. HTTP API (grouped)

JWT bearer auth on `/api/*`; roles enforced per route (ADMIN/MANAGER/VIEWER). Full request/response detail lives in [API.md](#).

- **Auth** — POST `/auth/register`, `/auth/login`, `/auth/logout`, `/auth/2fa/setup`, `/auth/2fa/enable`, `/auth/live/enable`, `/auth/live/disable`, `/auth/link/telegram`, `/auth/link/whatsapp`.
- **Bot control** — GET `/api/bot/state`; POST `/api/bot/start`, `/pause`, `/mode`, `/paper-forward`, `/emergency-stop`, `/emergency-reset`; PUT `/api/bot/live-settings`.
- **Trading** — GET `/api/trades`, `/api/trades/:id`; POST `/api/trades/manual`, `/api/trades/:id/approve`, `/api/trades/:id/reject`; POST `/api/positions/:ticket/close`, `/modify`; **journal** PUT | DELETE `/api/trades/:tradeId/journal`, GET `/api/journal`.
- **Scanner** — GET|PUT `/api/scanner`, POST `/api/scanner/run`.
- **Strategies** — GET|POST `/api/strategies`, PUT|DELETE `/api/strategies/:id`, GET `/api/strategies/presets`, `/validation-runs`; **lab** POST `/api/strategy-lab/run`, GET `/api/strategy-lab/last`.
- **Risk / exposure** — GET|PUT `/api/risk-settings`, GET `/api/exposure`, GET|PUT `/api/day-trading`.
- **Backtest** — POST `/api/backtest`, `/api/backtest/walk-forward`.
- **AI** — GET|PUT `/api/ai/provider`, PUT `/api/ai/provider-config`.
- **MT5** — GET `/api/mt5/accounts`, `/api/symbols`, `/api/market/:symbol`; POST `/api/mt5/connect`, `/api/mt5/accounts/:id/reconnect`.
- **News / sentiment** — GET `/api/news`, `/api/news/risk/:symbol`, POST `/api/news/refresh`, GET `/api/sentiment/:symbol`.
- **Copy** — GET|POST `/api/copy-traders`, **activate/deactivate/evaluate/signal**, PUT `/api/copy-`

`traders/:id/rules, GET /api/copied-trades.`

- **Ops / analytics** — GET /api/overview, /api/analytics, /api/audit, /api/notifications, PUT /api/notifications/prefs, /api/incidents (+ ack/resolve), /api/execution-comparisons, /api/paper-trades (+ performance).
- **Webhooks** — POST /webhooks/whatsapp (Twilio, form-encoded).
- **Health** — GET /health (bridge, AI, Redis, circuits — truthful ok + degraded[]).
- **Realtime** — GET /ws (WebSocket).

15. Background workers & coordination

`workers/scheduler.ts` sets up timers:

- **Analysis tick** — every 60 s. Runs a *coordinated cycle*: `protectiveTick` (always — reconcile paper trades, pre-news flatten, manage positions, equity guardian, day-trading exit, broker reconciliation) and `newTradeTick` (under a Redis lease — runs the scanner on its interval and evaluates every enabled strategy × symbol).
- **News refresh** — every `NEWS_REFRESH_MINUTES` (calendar + RSS headlines).
- **Strategy Lab** — weekly.
- **Floating P&L broadcast** — every 2 s.

Single-writer coordination — `scheduler-lease.ts` wraps new-trade work in a Redis lease so multiple backend processes don't double-trade. **Protection does not depend on the lease**: if Redis is down, new trades fail closed (`operationalTradingAvailable()` returns false) while direct-to-bridge position protection continues.

16. Realtime, notifications & connectors

- **WebSocket hub** (`ws/hub.ts`) — `broadcast(event, data)` fans out events (`bot_state`, `trade`, `paper_trade`, `approval_request`, `scanner`, `audit`, `incident`, `notification`, `emergency_stop`, `floating P&L`). The dashboard auto-reconnects and updates live without refresh.
- **Notifications** (`notifications/service.ts`) — per-user prefs route to web / Telegram / WhatsApp / email.
- **Telegram** (grammY) — approve/reject trades (`/approve_trade <id> [lots] [totp]`), status, and forwarded-signal copy ingestion.
- **WhatsApp** (Twilio) — inbound webhook commands (`approve trade <id> [lots]`).

17. Security & auth

- **JWT** (@fastify/jwt, default 30-min expiry) with role decorators (`authenticate`, `requireRole`). Passwords hashed with `bcryptjs`.
- **2FA (TOTP, otplib)** for live trading: `requireLiveTwoFactor` (master) and `autoLiveAuthorized` (standing consent so the AUTO bot can trade live without typing a code).

- **Token revocation without schema change** — `a token revoked before: <userId> epoch in` `SystemSetting`; JWT `iat` older than the cutoff is rejected. `POST /auth/logout` bumps it.
- **Rate limiting** — global 200/min; tighter per-route on `/auth/login` (10/min) and `/auth/register` (5/min).
- **Credentials at rest** — MT5 passwords AES-256-GCM encrypted ([lib/crypto.ts](#)); never stored plaintext.
- **Production config is fail-closed** — `config.ts` refuses to boot with placeholder secrets, default bridge key, or a provider selected without its API key/model.
- **Owner-scoped reads** — `/api/trades`, `/api/trades/:id`, `/api/overview` are scoped by `req.user.id` (operator action routes remain role-gated, not owner-scoped, by design for the shared-account single-admin deploy).

18. Resilience & operational health

- **Circuit breakers** ([lib/resilience.ts](#)) on MT5, AI, and news dependencies; state surfaced on `/health`.
- **Incidents** ([incidents/service.ts](#)) — dedupe-keyed, with min-interval throttling; raised for Redis-at-startup, circuit-open, equity-guardian, trading-unavailable; auto-resolved when the condition clears.
- **Fail-closed trading** — no new orders unless `emergencyStop=false`, `status=running`, and Redis is reachable.
- **Audit everything** — `AuditLog` captures auth, trade, risk, strategy, news, copy, system events; `ErrorLog` captures handled errors; API error handler never leaks internals.

19. Configuration (environment)

Validated in [config.ts](#) (zod). Highlights:

Var	Default	Purpose
PORT	4000	backend port
DATABASE_URL	—	Postgres (host port 5433 in dev)
REDIS_URL	redis://localhost:6379	leases, state mirror, circuits
JWT_SECRET	—	≥16 chars; rejected if placeholder in prod
CREDENTIALS_ENC_KEY	—	32-byte hex for AES-256-GCM
MT5_BRIDGE_URL / MT5_BRIDGE_API_KEY	localhost:5001	bridge endpoint + key
MT5 MOCK	true	mock vs real broker
AI_PROVIDER	ollama	ollama / anthropic / openai / openrouter

OLLAMA_URL / OLLAMA_MODEL / OLLAMA_TIMEOUT_MS	localhost:11434 / gemma3:12b / 60000	local model
ANTHROPIC_* / OPENAI_* / OPENROUTER_*	—	cloud providers (key + model)
NEWS_CALENDAR_URL / NEWS_RSS_FEEDS / NEWS_REFRESH_MINUTES	ForexFactory + ForexLive/ FXStreet / 15	news sources
WEB_SEARCH_PROVIDER / WEB_SEARCH_API_KEY	tavily / ""	Strategy Lab web research (empty = off)
TELEGRAM_BOT_TOKEN / TELEGRAM_ALLOWED_IDS	—	Telegram connector
TWILIO_*	—	WhatsApp connector
SMTP_* / EMAIL_FROM	—	email notifications

STRATEGY_VALIDATION_APPROVED and REQUIRE_2FA exist for legacy/config tests; the live posture is now driven by bot_state toggles (see runtime state below).

Runtime state lives in SystemSetting, not env:

- bot_state: status (stopped/running/paused/emergency_stop), mode, emergencyStop, demoMode, liveTradingEnabled, paperForward, requireLiveTwoFactor, autoLiveAuthorized. **Cached in-process with no TTL** — mutate it through the bot API/setBotState, not raw SQL, or the running process won't see the change until restart.
- scanner: scanner config. ai_provider: active LLM. day_trading: intraday cutoff.

20. Running locally

```
# 1. infra
docker compose up -d postgres redis

# 2. MT5 bridge (mock — no terminal needed)
cd mt5-bridge && MT5 MOCK=true python main.py           # :5001
# ...or real mode under Wine, attached to the MT5 terminal, MT5 MOCK=false

# 3. backend
cd backend && npm install && npm run prisma:generate
npm run seed           # seeds admin@example.com / changeme123
npm run dev            # :4000 (tsx watch)

# 4. frontend
cd frontend && npm install && npm run dev              # :3000
```

Verification: npm run verify (typecheck + tests) in backend/ and frontend/. GET /health should report the bridge connected, the AI reachable, Redis up, and all circuits closed.

A change to OLLAMA_MODEL (or any env) requires a **backend restart** — tsx watch only reloads .ts files, not the process environment.

21. Known limitations (be honest with operators)

- **No proven edge** in the bundled strategies (see §1 and the forensic audit). The system is built to *prove this rigorously and lose slowly under tight risk*, not to print money.
- **Netting attribution** — a netting account with multiple bot orders on one symbol is inherently 1:N at the broker and not perfectly per-order attributable.
- **Scanner does not enforce one-position-per-pair** (only the strategy pipeline does); it can stack same-pair positions, bounded by exposure caps and `maxPerDay`.
- **bot_state in-process cache has no TTL** — always change it via the API.
- **Small accounts** make tight per-trade risk caps mathematically unsatisfiable at the 0.01-lot floor; raise the cap or fund the account rather than expecting trades.
- **Single-admin RBAC** — operator routes are role-gated but not owner-scoped; full multi-tenant RBAC is a separate piece of work.